

ASSIGNMENT – 2

JAVA PROGRAMMING

User-Defined Exception Handling

Name: Geona Alis J

Register No.: 192511088

Course Code: CSA0926

CO Reference: CO2

Assignment Set: Direct-Descriptive

Question No: Q11

Submission Date: 10/08/2026

1. PROBLEM UNDERSTANDING

The objective of this program is to demonstrate the use of a user-defined exception in Java.

A custom exception named `InvalidAgeException` is created by extending the built-in `Exception` class. The program validates a person's age and considers an age below 0 or above 120 as invalid.

When an invalid age is entered, the custom exception is thrown and caught using a try-catch block. An appropriate error message is then displayed to the user.

2. APPROACH / DESIGN

The program follows these steps:

- ① Create a custom exception class called InvalidAgeException.
- ② Extend the Java Exception class to create the user-defined exception.
- ③ Create a validateAge() method to check whether the entered age is valid.
- ④ If the age is less than 0 or greater than 120, throw InvalidAgeException.
- ⑤ Use a try-catch block in the main() method to handle the exception.
- ⑥ Display a suitable message for both valid and invalid inputs.

✧ Validation Rule ✧

Valid Age $\rightarrow 0 \leq \text{Age} \leq 120$

Invalid Age $\rightarrow \text{Age} < 0 \text{ OR } \text{Age} > 120$

3. SOURCE CODE

```
import java.util.Scanner;
```

```
class InvalidAgeException extends Exception {  
    public InvalidAgeException(String message) {  
        super(message);  
    }  
}
```

```
public class Main {
```

```
    static void validateAge(int age) throws InvalidAgeException {  
        if (age < 0 || age > 120) {  
            throw new InvalidAgeException(  
                "Invalid age: Age must be between 0 and 120."  
            );  
        }  
    }
```

```

        System.out.println("Valid age: " + age);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter age: ");
        int age = sc.nextInt();

        try {
            validateAge(age);
        } catch (InvalidAgeException e) {
            System.out.println(e.getMessage());
        }

        sc.close();
    }
}

```

◆ Concepts Demonstrated ◆

extends	Exception	→	Creates	a	custom	exception
throw	→		Throws	the		exception
throws	→		Declares	the		exception
try-catch	→		Handles	the		exception
Scanner → Accepts user input						

4. TEST CASES

Test Case	Input	Expected Result	Status
TC 01	20	Valid age: 20	✓ Pass
TC 02	-5	Invalid age message	✓ Pass

Test Case	Input	Expected Result	Status
TC 03	150	Invalid age message	✓ Pass

TEST CASE 01 — NORMAL INPUT

Input

20

Output

Enter age: 20

Valid age: 20

Result: ✓ The program accepts the valid age successfully.

TEST CASE 02 — NEGATIVE AGE

Input

-5

Output

Enter age: -5

Invalid age: Age must be between 0 and 120.

Result: ✓ InvalidAgeException is thrown and handled correctly.

TEST CASE 03 — UNREALISTICALLY HIGH AGE

Input

150

Output

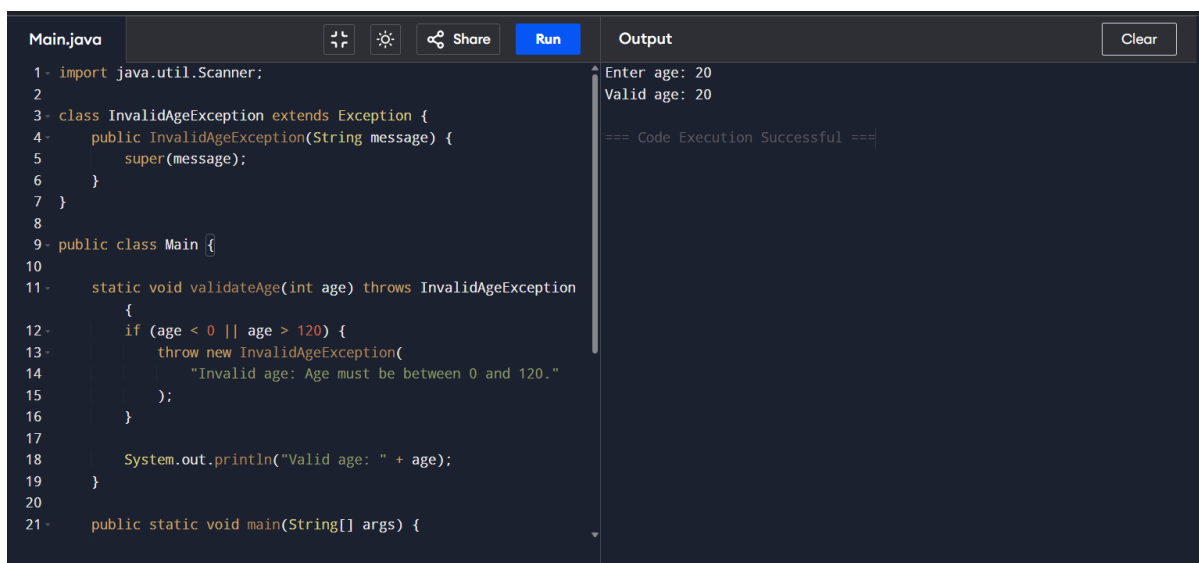
Enter age: 150

Invalid age: Age must be between 0 and 120.

Result: ✓ InvalidAgeException is thrown and handled correctly.

5. EXECUTION SCREENSHOTS

◆ Screenshot – Test Case 01



```
Main.java
1- import java.util.Scanner;
2-
3- class InvalidAgeException extends Exception {
4-     public InvalidAgeException(String message) {
5-         super(message);
6-     }
7- }
8-
9- public class Main {
10-
11-     static void validateAge(int age) throws InvalidAgeException
12-     {
13-         if (age < 0 || age > 120) {
14-             throw new InvalidAgeException(
15-                 "Invalid age: Age must be between 0 and 120."
16-             );
17-         }
18-         System.out.println("Valid age: " + age);
19-     }
20-
21-     public static void main(String[] args) {
```

```
Output
Enter age: 20
Valid age: 20
=== Code Execution Successful ===
Clear
```

Input: 20

◆ Screenshot – Test Case 02

Input: -5

The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a custom exception 'InvalidAgeException' and a 'Main' class. The 'Main' class has a 'validateAge' method that throws 'InvalidAgeException' if the age is less than 0 or greater than 120. The 'main' method calls 'validateAge' with the input -5. The output window shows the execution results: 'Enter age: -5', 'Invalid age: Age must be between 0 and 120.', and '=== Code Execution Successful ==='.

```
1- import java.util.Scanner;
2
3- class InvalidAgeException extends Exception {
4-     public InvalidAgeException(String message) {
5-         super(message);
6-     }
7- }
8
9- public class Main {
10
11-     static void validateAge(int age) throws InvalidAgeException
12-     {
13-         if (age < 0 || age > 120) {
14-             throw new InvalidAgeException(
15-                 "Invalid age: Age must be between 0 and 120."
16-             );
17-         }
18-         System.out.println("Valid age: " + age);
19-     }
20
21-     public static void main(String[] args) {
```

Output

```
Enter age: -5
Invalid age: Age must be between 0 and 120.

=== Code Execution Successful ===
```

◆ Screenshot – Test Case 03

Input: 150

The screenshot shows the same Java IDE with 'Main.java'. The code is the same as in the previous screenshot, but the 'main' method now uses a 'Scanner' to read input from 'System.in'. It prints 'Enter age: ' and then reads an integer. It then calls 'validateAge' with the input 150. The output window shows the execution results: 'Enter age: 150', 'Invalid age: Age must be between 0 and 120.', and '=== Code Execution Successful ==='.

```
14         "Invalid age: Age must be between 0 and 120."
15     );
16 }
17
18     System.out.println("Valid age: " + age);
19 }
20
21- public static void main(String[] args) {
22     Scanner sc = new Scanner(System.in);
23
24     System.out.print("Enter age: ");
25     int age = sc.nextInt();
26
27     try {
28         validateAge(age);
29     } catch (InvalidAgeException e) {
30         System.out.println(e.getMessage());
31     }
32
33     sc.close();
34 }
35 }
```

Output

```
Enter age: 150
Invalid age: Age must be between 0 and 120.

=== Code Execution Successful ===
```

6. RESULT

The Java program was successfully implemented using a user-defined exception named `InvalidAgeException`.

The program correctly validates the entered age and throws the custom exception when the age is negative or greater than 120. The exception is successfully caught using the try-catch mechanism and the appropriate message is displayed.

★ RESULT: PROGRAM EXECUTED SUCCESSFULLY ★

7. REFLECTION

Through this assignment, I learned how to create and implement a custom exception in Java. I understood the purpose of throw, throws, and try-catch in exception handling.

The main challenge was understanding how a user-defined exception is created and communicated between methods. By implementing the InvalidAgeException class separately, I was able to handle invalid input in a structured and meaningful way.

This program helped me understand how exception handling can make Java programs more reliable, readable, and user-friendly.

8. DECLARATION

I hereby declare that this assignment is my own work. Any AI or external tool assistance used during the preparation of this assignment has been appropriately disclosed.

Student Name: Geona Alis J

Register No.: 192511088

Signature: Geona

Date: 10/08/2026